

Laboratorio: AVL Tree y Union-Find — Análisis de Trade-offs

Sección 1 — Resultados Empíricos

Se realizaron pruebas de rendimiento en el código usando **JMH (Java Microbenchmark Harness)**, obteniendo mediciones exactas del flujo de operaciones de lectura y modificación bajo estrés en la JVM.

Configuración del Benchmark:

- **JVM:** OpenJDK 64-Bit Server VM, JDK 21.0.8 (con JIT y Compiler Blackholes)
- **Warmup:** 3 iteraciones de 1 segundo (ajustado empíricamente a 1 para lecturas exploratorias)
- **Measurement:** 5 iteraciones de 2 segundos (ajustado en perfiles continuos)
- **Forks:** 1
- **Métrica:** Throughput (Operaciones por milisegundo - ops/ms)
- **Conjunto de Datos:** $N = 100,000$ (para árboles) y $250,000$ aristas para grafos.

Estructura de Datos	Carga de Trabajo	Throughput Medido	Comportamiento
AVLTree (Propio)	Inserción (Insert)	0.032 ops/ms	Costoso por rebalanceo estricto.
TreeMap (JDK)	Inserción (Put)	0.044 ops/ms	+ 37.5% de rendimiento vs AVL.
AVLTree (Propio)	Búsqueda (Contains)	4,536.876 ops/ms	Ligeramente superior bajo estrés.
TreeMap (JDK)	Búsqueda (Get)	4,421.987 ops/ms	Extremadamente rápido (JIT optimizado).
DSU (Optimizado)	Agrupación (Union)	0.319 ops/ms	Rendimiento excelente.
HashMap (Naive)	Agrupación (Union)	$\sim 10^{-5}$ ops/ms	Cuelgues severos por GC y degradación.

Nota: Los resultados de HashMap ingenuo fueron tan drásticamente bajos que generaron advertencias de timeout en mediciones de calentamiento dentro de JMH (tiempos menores a la unidad de medida trivial).

Sección 2 — Análisis de Discrepancias

AVLTree vs. TreeMap (Red-Black Tree)

Los datos muestran una discrepancia interesante: el TreeMap de Java vence a nuestra implementación de AVLTree en inserciones considerables, pero pierde por un ligero margen en búsquedas (4536 ops/ms vs 4422 ops/ms).

1. **Relajación de Altura en TreeMap:** El TreeMap de Java está implementado en la base de un **Red-Black Tree**. Esta estructura garantiza que ningún camino es más del doble

de largo que cualquier otro, pero NO es un balanceo perfecto como el del AVL (cuyo factor de balance restringe a altura $\leq 1.44 \log_2 n$). Por tanto, **al insertar masivamente, el TreeMap gasta menos CPU en rotaciones** que el AVL, justificando su clara ventaja de \$0.044\$ frente a \$0.032\$ ops/ms.

2. **Eficiencia en Lectura:** El balanceo estricto del AVLTree probó su teoría en el benchmark: al estar estructuralmente más comprimido (achaparrado), los recorridos (loops en `contains`) toman métricamente menos iteraciones, dándole la ventaja en lectura.
3. **Optimizaciones Internas (JIT e Inline Cache):** TreeMap lleva décadas en la JDK. Las rutinas internas de C2 Compiler y *Method Inlining* de la JVM están virtualmente "acostumbradas" a sus patrones de acceso, lo cual amortigua bastante cualquier deficiencia que hipotéticamente un código Java normal sufriría.

DSU vs. Enfoque "Naive" (HashMap)

La humillante derrota del HashMap iterativo para resolver componentes conexas no es sorpresiva bajo la teoría analítica computacional:

1. **Locality y Manejo de Primitivos:** La implementación de DSU usa arreglos primitivos (`int[]`). El hardware moderno cachea (L1 y L2) la memoria contigua excepcionalmente bien. Por el contrario, un `HashMap<Integer, Integer>` demanda **Autoboxing/Unboxing** creando miles de objetos desperdigados en el Heap, desencadenando *pausas de Garbage Collection* y destruyendo el *Spatial Locality*.
2. **Path Splitting vs "Linked List" $O(N)$:** El HashMap sin compresión sufre rastreos $O(N)$ en árboles degenerados (se forman cadenas tipo lista enlazada al hacer clusters de grafos). El DSU usa *Union by Size* y *Path Splitting* (variante de Path Compression sin segunda pasada iterativa), llevando el costo asintótico real a la inversa de Ackermann $\alpha(N) \approx O(1)$.

Sección 3 — Recomendaciones de Diseño

Escenario Hipotético

Se debe diseñar un sistema de ranking de jugadores en tiempo real con millones de operaciones de inserción y consulta de top-K.

Recomendación de Estructura: TreeMap de la JDK

A partir de la experimentación científica en benchmarks y la robustez del diseño, la elección inequívoca es basarse en el patrón Red-Black Tree subyacente del TreeMap. Las razones se fundan en los siguientes hechos:

1. **Tolerancia a Millones de Inserciones:** Un sistema de *leaderboard* muta su estado a medida que los jugadores ganan o pierden puntuación. El benchmark demostró que el TreeMap lidera de forma holgada el *Throughput* en operaciones de guardado comparado con AVL, al evitar los reajustes estrictos de árbol completo en caso de cascadas LR / RL.
2. **Complejidad de Mantenimiento / Reducción de Bugs:** Emplear implementaciones canónicas elimina riesgos sistémicos de la plataforma, beneficiando la estabilidad de un proyecto en vivo.
3. **Operaciones Diferenciales $O(\log n)$:** Resolver Top-K es extremadamente sencillo con la interfaz `NavigableMap` integrada en TreeMap. Se puede recuperar subniveles

mediante funciones como `descendingMap().entrySet().stream().limit(K)`, algo que en código manual de AVL implicaría un mantenimiento tedioso de cursores pre-orden inversos.

4. **Resistencia de GC:** En millones de inserciones, las optimizaciones subyacentes del mapa de Oracle son virtualmente inigualables sin técnicas como JNI, logrando el compromiso ideal para operaciones asincrónicas.

En cuanto al ecosistema de Grafos/Matchmaking (e.g. equipos/lobbies), se recomendaría acoplar este ranking empleando estrictamente primitivas DSU, dado su ratio diferencial superador ante colecciones clásicas para agrupaciones lógicas.